

For example, consider a sales system with the following structure:

- Entities
 - Products
 - Orders
 - Customers
- Attributes
 - In Products (ProductID, ProductName, Price)
 - In Orders (OrderID, OrderDate)
 - In Customers (CustomerID, CustomerName, EmailAddress)
- Relationships
 - Customer places an order
 - An order includes one or more products

Conceptual schema is a model with insufficient details to build an actual database.

■ Logical schema

◆ **Logical schema:**
a detailed design of
the structure of tables
(fields and data types),
relationships between
tables and constraints.

Logical schema is a model that defines the structure of the database, including entities, attributes, data types, constraints, keys and relationships. It is a design that doesn't take into consideration the requirements of a specific database management system (DBMS). The logical schema is derived from the conceptual schema by:

- converting the entities into detailed tables
- defining the attributes by specifying the data types and constraints for each field in the table
- establishing primary and foreign keys
- defining relationships between the tables by using the keys
- normalizing the database to minimize data redundancy
- ensuring data integrity.

In the previous example:

Tables:

- Products
 - ProductID: INTEGER (PRIMARY KEY)
 - ProductName: VARCHAR
 - Price: REAL
- Orders
 - OrderID: INTEGER (PRIMARY KEY)
 - OrderDate: DATE
 - CustomerID: INTEGER (FOREIGN KEY)
 - PRODUCTID: INTEGER (FOREIGN KEY)
- Customers
 - CustomerID: PRIMARY KEY
 - CustomerName: VARCHAR
 - EmailAddress: VARCHAR, UNIQUE

Relationships:

- A customer places one or more orders (one-to-many).
- An order includes one or more products (one-to-many).

◆ **Physical schema:** an implementation of logical schema into a specific DBMS (database management system), showing how data is stored, indexed or accessed.

■ Physical schema

Physical schema includes specifics of storage devices, access methods, indexing, partitioning, access methods, views and configuration of the database on the storage media. It translates the logical schema into an implementation that fits the requirements of a specific database management system.

In the previous example:

Tables:

■ Products

- ProductID: INT PRIMARY KEY AUTO_INCREMENT
- ProductName: VARCHAR(100) NOT NULL
- Price: REAL NOT NULL

INDEX on ProductName for faster access based on the product name

■ Orders

- OrderID: INT PRIMARY KEY AUTO_INCREMENT
- OrderDate: DATE NOT NULL
- CustomerID: INT FOREIGN KEY NOT NULL
- ProductID: INT FOREIGN KEY NOT NULL

INDEX on CustomerID and ProductID for faster joins

■ Customers

- CustomerID: INT PRIMARY KEY AUTO_INCREMENT
- LastName: VARCHAR(100) NOT NULL
- FirstName: VARCHAR(100) NOT NULL
- EmailAddress: VARCHAR(100) NOT NULL UNIQUE

INDEX on LastName for faster access based on the last name

Storage parameters:

- Use indices described above for fast retrieval of data.
- Partition large tables like Orders by OrderID to improve query performance.

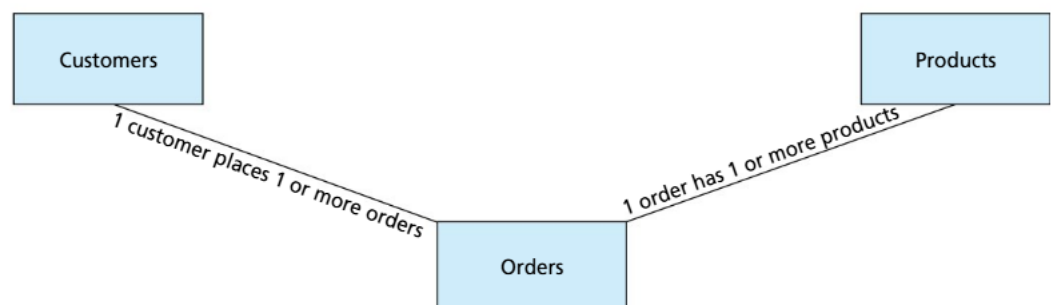
A3.2.2 Entity-relationship diagrams

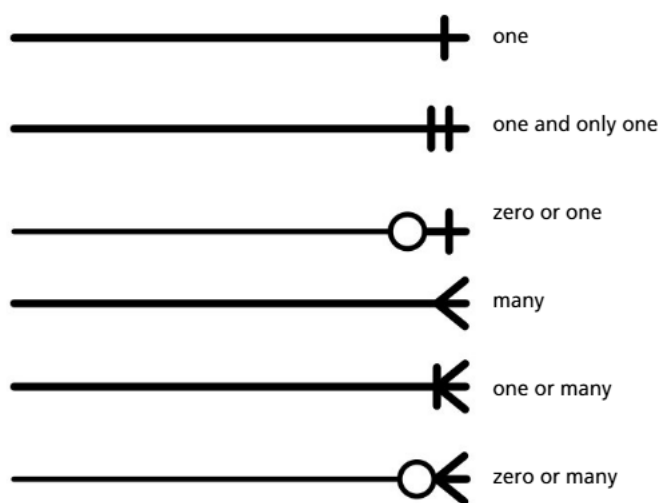
◆ **Entity-relationship diagram:** a visual representation of the entities in a database and the relationship between them.

An **entity-relationship diagram** (ERD) is a visual representation of the entities in the database and the relationship between them.

Besides providing a clear overview of the database structure, ERDs facilitate communication between stakeholders; act as documentation for the database design; support future development and maintenance of the database; and ensure data integrity and consistency through constraints and well-defined relationships.

For the sales system with Products, Orders and Customers entities, the ERD looks like this:





■ Modality of relationships

◆ **Modality:** the minimum number of instances of one entity that can be associated with an instance of another entity.

◆ **Cardinality:** the maximum number of times an instance in one entity can be associated with instances in the related entity.

The cardinality of relationships refers to the nature and extent of relationships between entities in an ERD. It specifies the number of instances of one entity that can or must be associated with each instance of another entity.

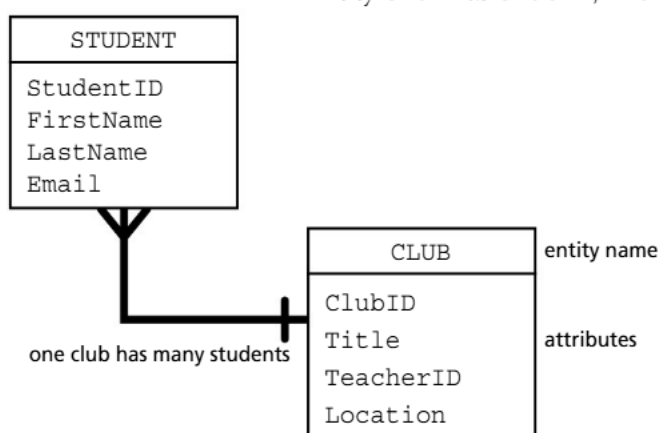
Cardinality refers to the maximum number of times an instance in one entity can be associated with instances in the related entity. It describes the “many” side of the relationship and it can be defined as:

- one-to-one
- one-to-many
- many-to-one
- many-to-many.

For example, consider a school management system that includes students and clubs as entities.

Entity STUDENT has StudentID, FirstName, LastName, Email as attributes.

Entity CLUB has ClubID, Title, TeacherID, Location as attributes.



Modality in ERDs refers to the minimum number of instances of one entity that can be associated with an instance of another entity. It defines whether the participation of an entity in a relationship is optional (0) or mandatory (1).

Consider an example involving data about patients and their medical records in a medical healthcare system. Most patients will have associated medical records, but new patients or newborn children might not have any medical history, therefore this is a type of optional relationship.

On the other hand, if you are to consider an e-commerce platform, every order must be associated with a customer (you cannot become a customer unless you place an order), so that is a type of mandatory relationship.

The relationship between the two entities can be represented as “a club has many students”.

Cardinality: one student can enrol in multiple clubs (one-to-many).

Modality: a club must have at least one student enrolled (mandatory for clubs); a student might not enrol in any clubs (optional for students).

Understanding both cardinality and modality is essential for accurately modelling the relationships and constraints in a database, ensuring it effectively reflects the real-world requirements and business rules.

ACTIVITY

Social skills: Support other students with application skills on practical tasks – help your classmates to analyse different database scenarios, identify entities, establish the appropriate relationships between tables and provide feedback to each other on how appropriate ERDs can be created.

A3.2.3 Data types used in relational databases

Data type for attributes	Description
CHARACTER	Fixed length text
VARCHAR(n)	Variable length text (n indicates the maximum number of characters)
INTEGER	Whole number
REAL	Number with a decimal part
DATE	Date as YYYY-MM-DD
TIME	Time as HH:MM:SS
BOOLEAN	True or False

Choosing the right data type is important for ensuring efficient indexing. For example, using `CHARACTER(8)` for fixed-length data like `UserID` is more efficient than using `VARCHAR(8)`, as this can lead to extra time during query execution due to the variable length storage. Also, the data type indicates the type of operations permitted. For example, if you store the quantity and price as fixed-length text, to perform calculations you will need to convert the text to integer or real values in the application, before using the data.

Another aspect of using appropriate data types is being able to store the data in the database into the corresponding field. If the type of data does not match the data type of the attribute in the database, the insertion attempt will throw errors.

Data consistency ensures users have access to up-to-date and accurate information, where all copies or instances are the same across all systems and database tables. Using different data types to refer to the same attribute on different platforms (database and application system) will lead to problems such as not being able to perform operations specific to the required data type, incorrect updates or queries.

A3.2.4 Constructing tables for relational databases

Properly defining the tables in a database supports the design of appropriate ERDs and ensures data integrity.

Considering a school management system, this could include the following tables:

■ `STUDENT (StudentID, FirstName, LastName, DateOfBirth, Email)`

■ `CLUB (StudentID, ClubTitle, TeacherName)`

■ `TEACHER (TeacherClub, Location)`

`STUDENT`

StudentID	FirstName	LastName	DateOfBirth	Email
101	Fatema	Kada	02/01/2010	f.kada@email.com
105	Alexandru	Buchidau	05/11/2009	a.buchidau@email.com
202	Kada	Hussein	07/25/2011	k.hussein@email.com

In the `STUDENT` table, the `StudentID` acts as a primary key to uniquely identify each record in the table.

CLUB

StudentID	ClubTitle	TeacherName
105	Robotics	Bobby Williams
202	Taekwondo	Dima White
101	Robotics	Bobby Williams
105	Arts and Crafts	Jane Doe

In the CLUB table, the StudentID is a foreign key (as it is a primary key in the STUDENT table). However, none of the fields in this table can act as a primary key, as they all have duplicates. But you could set the primary key to be a composite key, formed from the attributes StudentID and ClubTitle. On the other hand, you could add a new attribute ClubID to act as a primary key.

In case there is a need for a single field to act as primary key, it is possible to combine data from several attributes into one to act as a concatenated key.

TEACHER

TeacherClub	Location
Jane Doe Arts and Crafts	L101
Bobby Williams Robotics	H203
Dima White Taekwondo	B353

In the TEACHER table, the primary key is a concatenated key, formed from the attributes TeacherName and ClubTitle.

A3.2.5 Differences between normal forms

◆ **Normalization:** the process of organizing data in a relational database in a way to reduce data redundancy and to improve data integrity.

◆ **First normal form:** the status of a relational database in which entities do not contain repeating groups of attributes.

◆ **Atomic:** each attribute in a table containing indivisible values (values that cannot be broken down into more detailed sub-values).

Data **normalization** represents the process of organizing data in a relational database in a way to reduce data redundancy and to improve data integrity. Data redundancy is reduced as each item of data only occurs in one location in the database. This can reduce the possibility of update anomalies occurring, and it makes more efficient use of memory. Normalization leads to smaller tables with less information in each row, which leads to a reduction of input / output transfers, and so the CPU can work at full capacity since the likelihood of CPU activities being suspended is reduced. Normalization is achieved through a series of stages called “normal forms”, where each normal form has specific requirements for the table to be considered normalized at that level.

■ First normal form (1NF)

In **first normal form**, the table:

- has a primary key
- includes no duplicate attributes from the same table
- includes no repeated groups of attributes.

Therefore, you need to create separate tables for each group of related data, identifying each record by using the primary key, which is made of one single attribute or a set of attributes (composite or compound key), and ensure the entities do not contain repeated groups of attributes.

In 1NF, data in each field must be **atomic**. This means that each attribute contains indivisible values (values that cannot be broken down into more detailed sub-values). For example, an attribute called TeacherName in the TEACHER table is not an atomic field as this could

◆ **Functional dependency:** a relationship that exists between attributes, where one set of attributes (the determinant) determines the value of the other set (the dependent).

◆ **Full functional dependency:** where dependent attributes are determined by the determinant attributes.

◆ **Partial functional dependency:** when dependent attributes are partially determined by the determinant attributes.

◆ **Transitive dependency:** a type of functional dependency that occurs when a non-prime attribute is dependent on another non-prime attribute, rather than on the primary key.

◆ **Second normal form:** the status of a relational database in which entities are in 1NF and any non-key attributes depend upon the primary key.

◆ **Third normal form:** the status of a relational database in which entities are in 2NF and all non-key attributes are independent.

be further split into two different attributes called `LastName` and `FirstName`. Once this is achieved, the fields are atomic.

Atomicity ensures that each cell in the table will contain a single value, not complex structures like arrays or lists.

Functional dependency is a relationship that exists between attributes, where one set of attributes (the determinant) determines the value of the other set (the dependent). Typically, this is a relationship between the primary-key attribute and a non-key attribute. For example, in the `STUDENT` table, the `StudentID` (primary key and the determinant) determines the `FirstName`, `LastName`, `DateOfBirth` and `Email` values (the dependent). This means that, given the value of the `StudentID`, you can find the other details, but not vice versa. To ensure functional dependency in 1NF, you need to ensure entity atomicity and to remove repeating groups of attributes.

There are different types of functional dependencies:

- **Full functional dependency:** The dependent attributes are determined by the determinant attributes. For example, the `StudentID` fully determines the student's `FirstName`, `LastName`, `DateOfBirth` and `Email`.
- **Partial functional dependency:** The dependent attributes are partially determined by the determinant attributes. For example, the `StudentID` could partially determine the `FirstName`, `LastName` and `DateOfBirth` of the student, but not their course instructor for a club.
- **Transitive dependency:** The dependent attributes are determined by a set of attributes that are not included in the determinant attributes. For example, in an `EMPLOYEES` table, the `EmployeeID` may determine the `EmployeeDepartment`, which in turns determines their salary.

■ Second normal form (2NF)

In **second normal form** (2NF):

- entities are in 1NF
- any non-key attributes are fully functionally dependent on the primary key; there are no partial dependencies.

Partial-key dependency occurs in a table that has a composite key as primary key and one or more non-key attributes are dependent on only a subset of the composite primary key, rather than on the entire composite key. For example, in the `CLUB` table, the non-key attribute `TeacherName` is dependent on the `ClubTitle`, but not on the `StudentID`. As the primary key in this table is a composite key formed of both the `ClubTitle` and the `StudentID` fields, the `TeacherName` should have been fully functionally dependent on these two fields.

■ Third normal form (3NF)

In **third normal form** (3NF):

- entities are in 2NF
- all non-key attributes are independent (remove columns that are not fully functionally dependent on the primary key); the table contains no non-key dependencies.

Non-key or transitive dependency is a type of functional dependency that occurs when a non-prime attribute is dependent on another non-prime attribute, rather than on the primary key.

If the CLUB table looked like the one below, the primary key in the table would be ClubID. The ClubTitle is fully functionally dependent on the ClubID; however, the TeacherLastName is dependent on the TeacherID, which is not a primary key in the table.

CLUB

ClubID	ClubTitle	TeacherID	TeacherLastName
105	Robotics	1	Williams
202	Taekwondo	2	White
105	Robotics	1	Williams
106	Arts and Crafts	4	Doe

To resolve this non-key dependency, the table should be split into two: one storing club details (ClubID, ClubTitle and TeacherID) and the other teacher details (TeacherID and TeacherLastName).

CLUBDETAILS

ClubID	ClubTitle	TeacherID
105	Robotics	1
202	Taekwondo	2
106	Arts and Crafts	4

TEACHERDETAILS

TeacherID	TeacherFirstName	TeacherLastName
1	Bobby	Williams
2	Dima	White
4	Jane	Doe

Normalization issues can encompass data duplication, missing data and a range of dependency concerns, including data dependencies, composite key dependencies, transitive dependencies and multi-valued dependencies. For example, a car manufacturer produces two colours (black and grey) of each model every year. The attributes Colour and ManufacturingYear are dependent on the field CarModel, but they are independent of each other. Therefore, they can be called “multi-valued dependencies” on the CarModel.

Multi-valued dependencies occur when two attributes in a table are independent of each other, but both depend on a third attribute. This is important for achieving fourth normal form (4NF), which addresses certain types of redundancy not handled by earlier normal forms.

A3.2.6 Normalized databases (3NF)

Consider a library management system that stores the data in a table called “books”:

BOOKS

BookID	AuthorID	Author	Title	Pages	ProofReader
1	101	Boris Brown	History of AI	353	Amanda
2	102	Chris Joe	The Great G	200	Hamilton
3	19	Danny Bill	Big Tonny	190	Juan
5	101	Boris Brown	Amazing Future	399	Amanda

Normalizing this database to 3NF means:

1 Normalize it to 1NF:

- ☐ Set BookID as the primary key.
- ☐ Split the author into two different attributes: AuthorFirstName and AuthorLastName.

BookID	AuthorID	AuthorFirstName	AuthorLastName	Title	Pages	ProofReader
1	101	Boris	Brown	History of AI	353	Amanda
2	102	Chris	Joe	The Great G	200	Hamilton
3	19	Danny	Bill	Big Tonny	190	Juan
5	101	Boris	Brown	Amazing Future	399	Amanda

2 Normalize it to 2NF:

- ☐ Entities are in 1NF.
- ☐ There are no partial dependencies.

AuthorFirstName and AuthorLastName are dependent on AuthorID, while Title, Pages and ProofReader are dependent on the primary key (BookID). Therefore, we need to split this table as follows:

BOOKS

BookID	AuthorID	Title	Pages	ProofReader
1	101	History of AI	353	Amanda
2	102	The Great G	200	Hamilton
3	19	Big Tonny	190	Juan
5	101	Amazing Future	399	Amanda

AUTHOR

AuthorID	AuthorFirstName	AuthorLastName
101	Boris	Brown
102	Chris	Joe
19	Danny	Bill

3 Normalize it to 3NF:

- ☐ Entities are in 2NF.
- ☐ There are no transitive dependencies.

The ProofReader field has repeating values, and it is not necessarily fully functionally dependent on the BookID. Therefore, to remove non-transitive dependencies, you can create a new table for proof readers.

BOOKS

BookID	AuthorID	Title	Pages
1	101	History of AI	353
2	102	The Great G	200
3	19	Big Tonny	190
5	101	Amazing Future	399

AUTHOR

AuthorID	AuthorFirstName	AuthorLastName
101	Boris	Brown
102	Chris	Joe
19	Danny	Bill

PROOFREADERS

ProofReaderID	ProofReader
100	Amanda
222	Hamilton
123	Juan

Top tip!

Normalization of databases may be a challenging concept to grasp. Take the time to practise this concept; you can use past papers' exercises or create your own tables for this purpose.

Now, you need to link the BOOKS table with the PROOFREADERS table, so a new table is created.

BOOKS_PROOFREADERS

BookID	ProofReaderID
1	100
2	222
3	123
5	100

A3.2.7 Denormalizing databases

There are both advantages and disadvantages to normalizing and denormalizing databases. Overall, normalization plays a crucial role in designing efficient, maintainable and reliable databases that support data integrity and consistency while optimizing performance and scalability.

Normalization

Advantages	Disadvantages
Minimizes data redundancy Data is organized into separate tables for each entity, so it reduces data duplicates, saves storage space and ensures consistency.	Complexity in database schema Complex schema with multiple tables and relationships can make it difficult for developers to understand and maintain the database structure, especially in large or evolving systems.
Ensures data integrity Using specific rules regarding relationships and dependencies, normalization ensures insertion, update, delete queries and maintains data integrity.	Increased query complexity When needing to join multiple tables to perform queries, performance can drop.
Facilitates efficient data retrieval Well-defined relationships between tables support the development of efficient queries.	Increased storage requirements The increased number of tables and relationships may lead to higher storage requirements.
Supports scalability New data can be added without significantly altering the existing structure.	Not ideal for all use cases Normalization is based on relational database principles, and it may not fit all applications or types of data.

Advantages	Disadvantages
Promotes data consistency Eliminating data redundancy and defining clear relationships promotes data consistency.	Difficulty in balancing normalization When normalizing a database, you need to aim for a balance between reducing data redundancy and maintaining performance. Over-normalization (too many tables and relationships) or under-normalization (failing to separate data appropriately) may lead to maintenance issues and low performance.
Simplifies database maintenance Changes can be made to a single table without affecting other tables.	Overheads in updates Updating records may require changing data in different places, which can reduce performance or increase the complexity of update operations.

■ Denormalization

Denormalization refers to deliberately allowing for data redundancy in a database design to improve the performance of queries.

◆ **Denormalization:** deliberately allowing for data redundancy in a database design to improve the performance of queries.

Advantages	Disadvantages
Improved query performance There is less need for joins; the simpler structure of the tables improves the performance of read-heavy queries.	Data redundancy There is the possibility of inconsistencies if updates are not properly managed; it is more challenging to synchronize and maintain the data.
Simplified data retrieval Data is stored closer to the way it is accessed by applications so it allows for faster retrieval of data as there are less complex joins between tables.	Increased storage overhead Redundant data requires additional storage space, which can become significant in large or evolving systems.
Enhanced scalability This reduces the overhead of maintaining complex relationships, so it supports larger data sets and higher transaction volumes without sacrificing performance.	Maintenance challenges Managing denormalized databases requires careful planning and maintenance to ensure data integrity and consistency.

There are situations where denormalization can enhance performance, especially when read performance is crucial and outweighs concerns about data redundancy and updates complexity. Some specific scenarios are as follows:

Read-intensive applications

Read-intensive applications refer to applications in which the focus is on retrieving data, rather than updating it. By reducing the number of tables, relationships and joins, data can be accessed faster, improving query performance and response time.

Reporting and analytics

Reporting and analytics are used when generating complex reports or analysing large sets of data. Reducing the number of tables, simplifying relationships and decreasing the need for joins speeds up reporting and analytics queries.

Data warehousing

In data warehousing, the focus is on storing and analysing historical data from different sources. Simplifying complex queries across different data sources and using fewer tables to organize related data improves query performance and response time.

Denormalization simplifies query structures by reducing the need for joins between tables. Simpler queries are easier to write, faster to execute, easier to understand and easier to maintain. On the other side, allowing for data redundancy means increased storage

requirements, and increased complexity in maintenance and update operations. Therefore, there is a need to find a balance between the two, and this requires:

- analysing the specific requirements of the application (is the focus on reading or retrieving the data or on updating it?)
- identifying whether the benefits of denormalizing the database are higher than the risk of data redundancy and complexity (sometimes a partially denormalized schema might be a solution)
- implementing appropriate strategies to mitigate risks and optimize performance (implement robust data validation, monitor query performance and storage requirements, and so on).

ACTIVITY

Independent learning: Independent reflection and targets for improvement – identify your strongest and weakest points and set short-term goals for achieving success. When analysing your strongest and weakest points, consider: technical skills and soft skills; constructive or positive feedback from peers and teachers; successful projects; formative assessments; and knowledge gaps or technical gaps. Short-term goals will keep you focused; they will allow you to set an action plan that can be monitored and evaluated progressively.

Utilitarianism, the greatest good for the greatest number. The ends justify the means.

Utilitarianism is an ethical theory that suggests the best action is the one that maximizes overall happiness or well-being. This approach is often summarized by the phrase "the greatest good for the greatest number". According to this view, the moral value of an action is determined by its outcome or consequences, rather than by any intrinsic qualities of the action itself. For example, if sacrificing the well-being of a few individuals leads to a greater overall benefit for society, a utilitarian would argue that such a sacrifice is justified.

When managing databases, particularly those containing personal information, there's often a trade-off between privacy and utility. A utilitarian approach might justify the use of personal data without explicit consent if it leads to a greater good, such as improving public health through data-driven insights. During a pandemic, health authorities might access and analyse large data sets of personal health information to track the spread of the virus. From a utilitarian perspective, the potential benefits to society (for example, controlling the pandemic) might outweigh individual privacy concerns. The utilitarian principle of "the greatest good for the greatest number" provides a framework for making decisions about how databases are managed, used and secured.

Discuss how this approach may be balanced against other ethical considerations, such as individual rights and fairness, which may not always align with a purely utilitarian perspective.

REVIEW QUESTIONS

- 1 Define the term "schema".
- 2 Discuss the characteristics of a normalized database.
- 3 Identify two issues caused by data redundancy.
- 4 State the characteristics of:
 - a 1NF
 - b 2NF
 - c 3NF
- 5 Distinguish between conceptual and logical schema.

SYLLABUS CONTENT

By the end of this chapter, you should be able to:

- ▶ A3.3.1 Outline the difference between data language types within structured query language (SQL)
- ▶ A3.3.2 Construct queries between two tables in SQL
- ▶ A3.3.3 Explain how SQL can be used to update data in a database
- ▶ A3.3.4 Construct calculations within a database using SQL's aggregate functions (HL)
- ▶ A3.3.5 Describe different database views (HL)
- ▶ A3.3.6 Describe how transactions maintain data integrity in a database (HL)

A3.3.1 Data language types within SQL

Data language types include data definition language (DDL) and data manipulation language (DML).

■ Data definition language (DDL)

Data definition language (DDL) is used to create, modify and remove data structures from a relational database.

◆ **Data definition language:** language that is used to create, modify and remove data structures from a relational database.

SQL DDL instructions	Explanation
CREATE	Create a new database object (table, view, index)
PRIMARY KEY	Set a field as a primary key
FOREIGN KEY ... REFERENCES ...	Set a field as a foreign key by specifying the field and the table it is associated with
ALTER	Change the structure of an existing database object: alter a table structure, by adding or removing columns or adding constraints
DROP	Delete database objects (tables, indices, views)

CREATE statements

CREATE DATABASE:

```
CREATE DATABASE HOSPITAL
```

CREATE TABLE:

```
CREATE TABLE Employees (
  EmployeeID INT,
  DepartmentID INT PRIMARY KEY,
  LastName VARCHAR(20)
);
```